ADR-003: Modular Abstraction Layer

PROPOSED — FEBRUARY 2026

ARCHITECTURE DECISION RECORD

This Architecture Decision Record proposes a **Modular Abstraction Layer** for the agentic-workflows-v2 runtime — a swappable backend architecture that cleanly separates engine-agnostic AI concepts from framework-specific implementations. Authored by T. Freeman and the Architecture Team, this ADR addresses the growing complexity of managing multiple execution engines and the need to support a third backend: the Microsoft Agent Framework (RC 1.0.0rc2).

The core goal is to allow the same YAML workflow to run identically on any installed adapter — Native DAG, LangChain/LangGraph, or MS Agent Framework — without rewriting business logic, breaking existing tests, or forcing users to install dependencies they don't need.

Made with GAMMA

Context & Problem Statement

The agentic-workflows-v2 runtime currently implements two execution engines: the **Native DAG Engine** (`engine/`), a pure Python asyncio-based Kahn's algorithm scheduler with maximum parallelism, retry policies, hierarchical context, and checkpointing; and the **LangChain/LangGraph Engine** (`langchain/`), a LangGraph state machine compiler with streaming, checkpointing, and message-passing semantics. Both satisfy the same `ExecutionEngine` Protocol defined in `engine/protocol.py`, yet they are practically isolated from one another.

Hardcoded CLI & Server

The CLI and server hardcode LangChain — the native DAG engine is completely unreachable from the user surface.

Disconnected Tool Registration

Tools have two separate registration systems: `BaseTool` (native) and `@tool` (LangChain), with no unified bridge.

Tight Agent Coupling

Agents (`BaseAgent`) are tightly coupled to native engine imports, and memory exists only inside `BaseAgent.ConversationMemory` with no swappable protocol.

Split Configuration

Configuration is fragmented between `engine/step.py` and `langchain/config.py`, with no unified YAML-driven config layer.

Additionally, the Microsoft Agent Framework reached Release Candidate 1.0.0rc2 in February 2026, offering a third viable backend with enterprise Azure integration, built-in orchestration patterns, and governance features. The core problem: educational AI concepts (ReAct loops, DAG orchestration, Memory, Tool Use) are entangled with framework-specific implementations, preventing backend swappability and making it impossible to compare approaches or adopt new frameworks without rewriting business logic.

Made with GAMMA

Decision Drivers

01

Framework Independence

Core AI concepts must be expressible without any framework dependency whatsoever.

02

Zero Breaking Changes

All 452 existing tests must pass without modification via backward-compatible re-export shims.

03

Opt-in Complexity

Each adapter is an optional dependency group — users install only what they need.

04

Comparability

The same YAML workflow must run identically on any installed adapter.

05

Extensibility

Adding a new adapter (e.g., CrewAI, DSPy) requires implementing ~5 Protocol methods, not forking the codebase.

06

Production RAG Integration

The RAG pipeline (`tools/rag/`) must work as a tool/memory backend with ANY adapter.

Made with GAMMA

The Decision: Three-Tier Architecture

The decision is to implement a **Modular Abstraction Layer** organized into three distinct tiers: `/core` for engine-agnostic concepts, `/adapters` for framework implementations, and `/examples` for demonstrating swappability across adapters.

/core — Engine-Agnostic Concepts

Pure Python protocols and data structures with **zero framework dependencies**. Contains: ExecutionEngine Protocol, AgentProtocol, ToolProtocol, MemoryStore Protocol, ExecutionContext, DAG & StepDefinition, WorkflowConfig, and Contracts (StepResult, WorkflowResult, TaskInput/TaskOutput).

/adapters — Framework Implementations

Three adapters, each in its own package: **Native** (zero pip deps, Kahn's algorithm + asyncio), **LangChain** (requires langchain-core, langgraph — StateGraph compiler + tool bridge), and **MS Agent Framework** (requires agent-framework>=1.0.0rc2 — MAFExecutor + Azure integration).

/examples — Swappability Demos

Identical workflows with per-adapter runner scripts demonstrating that the same use case executes correctly on any installed backend without modification to business logic.

Made with GAMMA

Core Protocol Definitions

The four foundational protocols defined in `/core` form the contract that every adapter must satisfy. These are `@runtime_checkable` Python Protocols, enabling structural subtyping without inheritance.

ExecutionEngine

```
@runtime_checkable
class ExecutionEngine(Protocol):
    async def execute(
        self,
        config: WorkflowConfig,
        ctx: ExecutionContext | None = None,
        on_update: UpdateCallback | None = None,
        **kwargs: Any,
    ) -> WorkflowResult: ...
```

AgentProtocol

```
@runtime_checkable
class AgentProtocol(Protocol):
    name: str
    tier: int
    async def run(
        self,
        input: TaskInput,
        ctx: ExecutionContext
    ) -> TaskOutput: ...
```

ToolProtocol

```
@runtime_checkable
class ToolProtocol(Protocol):
    name: str
    description: str
    def execute(
        self,
        params: dict[str, Any]
    ) -> ToolResult: ...
```

MemoryStore Protocol

```
@runtime_checkable
class MemoryStore(Protocol):
    async def store(self, key: str,
        value: Any, *,
        namespace: str = "default"
    ) -> None: ...
    async def retrieve(self, key: str, *,
        namespace: str = "default"
    ) -> Any | None: ...
    async def search(self, query: str, *,
        top_k: int = 5
    ) -> list[dict[str, Any]]: ...
    async def delete(self, key: str, *,
        namespace: str = "default"
    ) -> bool: ...
```

Made with GAMMA

Adapter Registration, Tool Bridge & Concept Mapping

Adapters self-register via a singleton `AdapterRegistry` using a `try/except ImportError` pattern in each adapter's `__init__.py`. This means an adapter is only available if its dependencies are installed — graceful degradation by design. Usage is as simple as `engine = get_adapter("native")` or `engine = get_adapter("langchain")`.

The **Tool Bridge** ensures that registering a tool once in `core/ToolRegistry` makes it available to all adapters automatically. RAG tools (`RAGSearchTool`, `RAGIngestTool`) work identically across all backends.

Core Concept	Native	LangChain	MS Agent Framework
ReAct Loop	agent_resolver multi-turn tool loop (8 rounds, 12 calls/round)	create_react_agent()	Built into agent.run()
Tool Invocation	BaseTool.execute()	ToolNode in StateGraph	FunctionTool
Graph Scheduling	Kahn's algorithm (max parallelism)	LangGraph state machine	Superstep execution
Conditional Routing	StepDefinition.when predicate	Conditional edges	Conditional Edge
Short-term Memory	ConversationMemory sliding window	MemorySaver checkpoint	InMemoryHistoryProvider
Long-term Memory	MemoryStore protocol	LangMem	BaseContextProvider pipeline
Semantic (RAG)	RAGMemoryStore (tools/rag/)	RAG tool in ToolNode	ContextProvider + RAG
Tool Registration	ToolRegistry.register()	ALL_TOOLS list	agent.add_tool()

Made with GAMMA

Migration Strategy & Dependency Model

Strangler Fig Pattern

Existing packages (`engine/`, `langchain/`) become thin re-export shims. This means `from agentic_v2.engine import ExecutionContext` still works (backward-compat), while new code imports from `agentic_v2.core` (canonical path). Shims can emit `DeprecationWarning` in a future release. All 452 existing tests pass without modification.

```
# engine/context.py (after migration)
"""Backward-compatible re-exports."""
from agentic_v2.core.context import (
    EventType, ExecutionContext,
    ServiceContainer, get_context,
    set_context, reset_context,
)
```

Opt-in Dependency Groups

pip install -e "."

Core protocols + native adapter (zero framework deps)

pip install -e ".[langchain]"

Adds LangChain/LangGraph adapter

pip install -e ".[ms-agent]"

Adds Microsoft Agent Framework adapter

pip install -e ".[all-adapters]"

All adapters installed together

The native adapter is always available — it has no dependencies beyond stdlib and asyncio.

Made with GAMMA

CLI, Server Integration & RAG Pipeline

The CLI gains an `--adapter` flag enabling per-run backend selection, adapter listing, and cross-adapter comparison. The server exposes adapter selection via a query parameter on the workflow run endpoint.

CLI Usage

```
agentic run code_review \
  --input review.json \
  --adapter native

agentic run code_review \
  --input review.json \
  --adapter langchain

agentic list adapters

agentic compare code_review \
  --adapters native,langchain
```

Server API

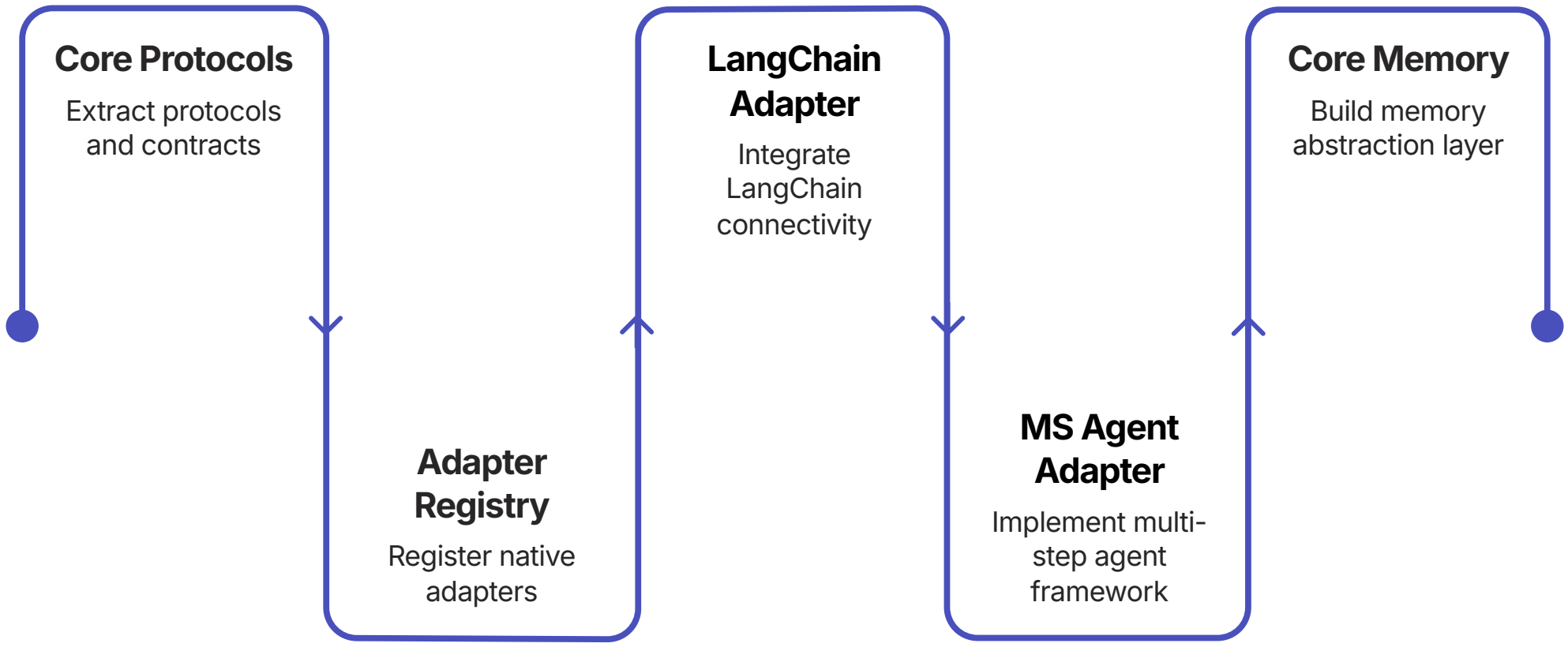
```
POST /api/workflows/{name}/run
Body: {
  "inputs": {...},
  "adapter": "langchain"
}

GET /api/adapters
Response: {
  "adapters": [
    "native",
    "langchain",
    "ms-agent"
  ]
}
```

RAG (`tools/rag/`) integrates at three levels with any adapter. **Level 1 (Tool Registration)**: `RAGSearchTool` and `RAGIngestTool` extend `BaseTool` and are converted automatically by each adapter's tool bridge. **Level 2 (Memory Backend)**: `RAGMemoryStore` implements the `MemoryStore` protocol, providing semantic search backed by the vector store. **Level 3 (Context Augmentation)**: A tier-0 RAG retriever step can be included in any YAML workflow via a `retrieve_context` step that feeds into downstream agents.

Made with GAMMA

Implementation Phases



The implementation is structured across eight phases, progressing from foundational protocol extraction through to user-facing CLI and example workflows. Each phase is independently testable and deployable, with new test coverage added at every step.

#	Phase	Scope	New Files	New Tests
1	Core Protocols & Contracts	Extract engine-agnostic protocols into core/, create backward-compat shims	~10	~20
2	Adapter Registry & Native Adapter	Wrap existing DAG engine as first adapter, introduce AdapterRegistry	~7	~12
3	LangChain Adapter	Wrap existing LangGraph engine as second adapter, tool bridge	~6	~15
4	MS Agent Framework Adapter	New adapter demonstrating extensibility, concept mapping	~6	~18
5	Core Memory Abstraction	Extract memory from BaseAgent into standalone protocol	~1	~10
6	RAG Integration Bridge	Connect tools/rag/ to all adapters via tool bridge and memory protocol	~3	~8
7	CLI & Server Adapter Awareness	Add --adapter flag, server routing, compare command	~3 modified	~8
8	Examples	Three workflows × three adapters demonstrating swappability	~12	—

Made with GAMMA

Consequences, Risks & File Summary

✅ Positive Consequences

- Same YAML workflow runs on any installed adapter — true backend swappability
- Adding new adapters requires implementing ~5 Protocol methods, not forking code
- Core concepts serve as a framework-agnostic teaching surface for AI patterns
- RAG tools work across all adapters via unified tool bridge
- Zero breaking changes via Strangler Fig migration — all 452 tests pass

⚠️ Negative Consequences

- Additional indirection layer (core → adapter → framework) increases cognitive overhead
- Tool bridge adds thin conversion overhead per tool call
- MS Agent Framework adapter depends on RC-status package (1.0.0rc2)
- Migration effort: ~4,800 lines total (~60% refactored, ~40% new)

Risk	Mitigation
MS Agent Framework API changes before GA	Pin to RC2; adapter is isolated — changes don't affect core
Performance overhead from abstraction	Tool bridge is ~microseconds per call; benchmarked before merging
Backward compatibility breakage	Strangler Fig shims + full existing test suite as regression gate
Adapter feature disparity	Protocol optional capabilities (SupportsStreaming, SupportsCheckpointing) allow graceful degradation

40

New Files

Across core/, adapters/, and examples/

10

Modified Files

Backward-compat shims, CLI, and server

4800

Total Lines

2,800 source + 800 examples + 1,200 tests

452

Existing Tests

All pass without modification — zero breaking changes

Made with GAMMA